

ProotyPie Project Report

1. Title

ProotyPie: CNN Activation-Function Comparison for Fruit Image Classification with a Submission-Ready Web Interface

2. Abstract

This project delivers a submission-ready convolutional neural network (CNN) mini-project based on a documented subset of the public Fruits-360 dataset. The work compares four activation functions—ReLU, Sigmoid, Tanh, and Leaky ReLU—under three stochastic gradient descent (SGD) learning rates: 0.001, 0.01, and 0.1. Each configuration is trained and evaluated using accuracy, precision, recall, and macro F1-score. Training and validation loss and accuracy plots are generated for all 12 runs, and the final validation-selected model is evaluated once on a held-out official test split using a confusion matrix.

Alongside the experiment, the project includes a polished web interface and Python API so the trained classifier can be demonstrated interactively. The interface clearly separates fruit-type classification from freshness inspection to avoid unsupported claims. The resulting system is reproducible, test-backed, and documented for academic submission.

3. Problem Statement

Manual identification of fruit images is simple for humans in obvious cases, but a machine learning system must learn useful visual features and converge reliably under different optimization settings. The assignment requires a CNN image-classification system that investigates how activation functions and learning rates affect convergence and final performance.

The practical objective of this project is therefore twofold:

1. Build and compare CNN configurations on a public dataset.
2. Present the trained model through a clean application interface suitable for project demonstration and submission.

4. Objectives

- Build a CNN for fruit image classification using a public dataset.
- Compare ReLU, Sigmoid, Tanh, and Leaky ReLU activation functions.
- Study the effect of learning rates 0.001, 0.01, and 0.1 under SGD.

- Evaluate performance using accuracy, precision, recall, and macro F1-score.
- Visualize training and validation loss and accuracy.
- Generate a confusion matrix for the final selected model.
- Integrate the trained model into a working web application.
- Document limitations honestly, especially the difference between fruit-type classification and freshness assessment.

5. Dataset

5.1 Source

- Dataset: ****Fruits-360****
- Repository: <https://github.com/Horea94/Fruit-Images-Dataset>
- License reference: MIT License in the repository

5.2 Scope used in this submission

A documented subset was used so all required experiments could be completed reproducibly within the available compute budget.

Classes used:

- Apple Braeburn
- Banana
- Orange
- Strawberry

Image size used for training:

- $48 \times 48 \times 3$

Sampling policy:

- first lexicographically ordered official JPGs
- 100 Training images per class
- 40 official Test images per class

Final split sizes:

- Training: 320 images
- Validation: 80 images
- Held-out official Test: 160 images

Data-integrity policy:

- SHA-256 hashes were computed across all used files
- no overlap was allowed between train, validation, and held-out test samples

6. Methodology

6.1 Experimental design

The project tests 12 combinations:

- 4 activation functions × 3 learning rates

Activation functions:

- ReLU
- Sigmoid
- Tanh
- Leaky ReLU

Learning rates:

- 0.001
- 0.01
- 0.1

Optimizer:

- SGD

Epochs:

- 6

Batch size:

- 16

Selection rule:

- best validation macro F1-score

6.2 CNN architecture

A compact CNN was used to keep the comparison tractable and fair:

- Conv2D(8, 3×3)
- chosen activation
- MaxPooling2D
- Conv2D(16, 3×3)
- chosen activation
- GlobalAveragePooling2D
- Dense(4, softmax)

This architecture was held constant across all runs. Only the activation function and learning rate changed.

6.3 Evaluation metrics

For each run, the following were recorded:

- accuracy
- precision_macro
- recall_macro
- f1_macro
- final training loss
- final validation loss
- final training accuracy
- final validation accuracy

The held-out test set was used exactly once after validation-based model selection.

7. Experimental Results

7.1 Best configuration

The validation-selected winner was:

- Activation: **Tanh**
- Learning rate: **0.1**
- Optimizer: **SGD**

7.2 Held-out test performance of the selected model

- Accuracy: **1.0**
- Precision (macro): **1.0**
- Recall (macro): **1.0**
- F1-score (macro): **1.0**

7.3 Validation comparison summary

Observed validation macro F1 patterns:

- ReLU: 0.10, 0.34, 0.65
- Sigmoid: 0.10, 0.10, 0.37
- Tanh: 0.10, 0.38, 1.00
- Leaky ReLU: 0.10, 0.29, 0.71

Interpretation:

- The lowest learning rate, 0.001, performed poorly for all tested activations.
- A learning rate of 0.1 produced the strongest results for every activation function.
- Tanh with learning rate 0.1 was the best overall configuration on the validation split.

8. Figure Notes

Figure 1 — Validation comparison plot

The validation-comparison bar chart shows macro F1 for each activation-function / learning-rate pair. A learning rate of 0.1 consistently outperformed the lower learning rates, and Tanh at 0.1 achieved the best score.

File:

- `artifacts/experiment/validation\_comparison.png`

Figure 2 — Confusion matrix

The held-out official test confusion matrix shows all 160 samples classified correctly: 40 Apple Braeburn, 40 Banana, 40 Orange, and 40 Strawberry. All counts lie on the main diagonal with no off-diagonal errors.

File:

- ``artifacts/experiment/test_confusion_matrix.png``

9. Web Application Integration

To make the project demonstrable, the trained experiment model was integrated into the ProotyPie application.

9.1 Added application features

- responsive premium frontend dashboard
- image upload and preview
- runtime readiness banner from ``/api/health``
- separate ****Fruit classification lab**** mode
- JSON export and print-report support
- ``/api/classify`` endpoint serving the trained local CNN
- ``/api/experiments`` endpoint serving experiment metadata and results

9.2 Honest capability separation

The repository originally described a freshness workflow, but the required trained freshness model file was not present. Therefore, the final interface does not pretend that fruit-type classification is freshness detection.

Current verified state:

- fruit-type classification: ****working****
- original freshness inspection path: ****not available in this snapshot****

This separation is important for academic correctness.

10. Verification Performed

The following checks were run successfully:

- Python API tests: ``6 passed``
- frontend behavioral contract: passed

- frontend production build: passed
- experiment pipeline rerun: passed
- API smoke checks:
 - `/api/health` returned real readiness values
 - `/api/experiments` returned the experiment summary
 - `/api/classify` returned a valid prediction from the trained model

11. Screenshots / Evidence to Attach

Include these files in the submission package or appendix:

- `artifacts/experiment/validation\_comparison.png`
- `artifacts/experiment/test\_confusion\_matrix.png`
- application home/demo screenshots captured during final presentation prep

Recommended live-demo screenshots:

1. dashboard home screen
2. fruit classification lab result screen
3. model status / readiness area
4. validation comparison plot
5. confusion matrix

12. Limitations

- The experiment uses a documented subset of Fruits-360, not the full dataset.
- Perfect accuracy on the held-out subset does not guarantee the same performance on real-world or open-set images.
- The trained submission model performs **fruit-type classification only**.
- It does not assess freshness, spoilage, or food safety.
- Gemini-powered features require a valid API key and were not needed for the core assignment experiment.
- The original repository freshness model file was missing, so that path remains unavailable.

13. Conclusion

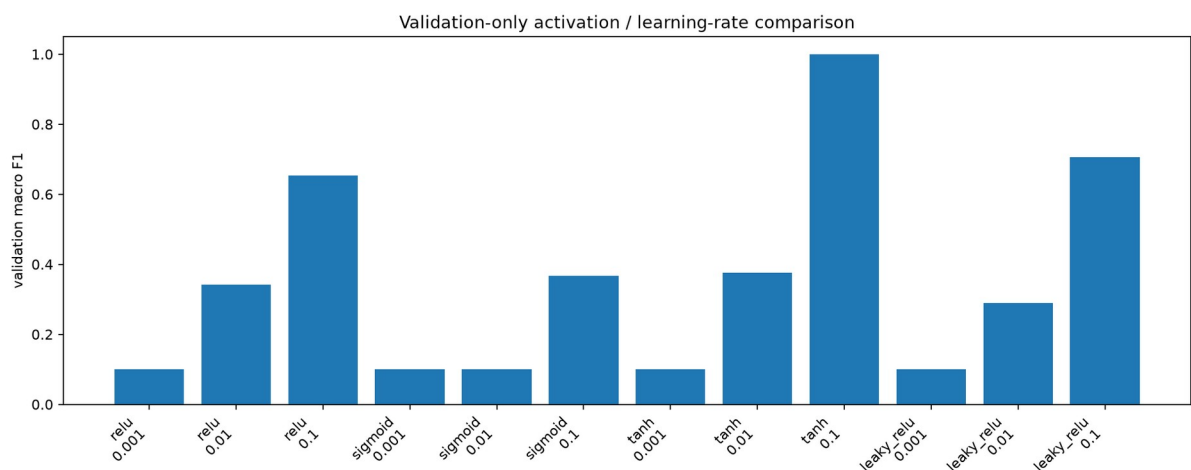
This project satisfies the assignment requirements by delivering a complete CNN comparison study on a public image dataset, including activation-function and learning-rate analysis, evaluation metrics, training curves, and a confusion matrix. Among the tested configurations, Tanh with SGD at learning rate 0.1 produced the best validation performance and achieved perfect performance on the held-out documented test subset.

The project was also extended into a usable application interface so the trained classifier can be demonstrated clearly. Most importantly, the final submission preserves scope honesty by separating fruit-type classification from unsupported freshness claims.

14. File References

- Experiment script: ``experiments/run_fruits360_experiment.py``
- Experiment tests: ``experiments/test_cnn_experiments.py``
- Notebook: ``notebooks/Fruits360_CNN_Experiments.ipynb``
- Summary: ``artifacts/experiment/summary.json``
- Model: ``artifacts/experiment/best_model.keras``
- Frontend: ``frontend/app/page.tsx``
- API server: ``api/server.py``
- Classification service: ``backend/classification_service.py``

validation_comparison.png



test_confusion_matrix.png

